**MBEYA UNIVERSITY OF SCIENCE AND TECHNOLOGY**

**COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**BACHELOR DEGREE IN COMPUTER SCIENCE**

**UQF LEVEL 8**

**FINAL PROJECT REPORT**

# PROJECT TITLE: DEVELOPMENT OF A DIGITAL AGRICULTURAL KNOWLEDGE LIBRARY FOR SMALLHOLDER FARMERS

**PROJECT TYPE: PROBLEM SOLVING**

**GROUP NAME: ICTbello**


| NAME OF CANDIDATES | ADMISSION NO. |
|---|---|
| ABUBAKARI A. SHAWA | CA/BCS/25/10047 |

## INTRODUCTION

Smallholder farmers suffer developing regions often face a significant information gap. While like M-Kilimo modern agricultural app, pest management strategies, and climate-smart farming practices exist, they are frequently trapped in academic papers or stored in centralized government databases that are inaccessible to the average farmer.

Current challenges include:

- Fragmentation of Information: Agricultural advice is scattered across various platforms, making it difficult to find cohesive guides.
- Lack of Localization: Existing digital tools often fail to provide information tailored to the specific crops and soil types relevant to small-scale local farming.
- Mobile Accessibility: Many existing resources are not optimized for mobile devices, which are the primary gateway to the internet for most smallholder farmers.

## MOTIVATION

The motivation behind this project is to democratize agricultural expertise. By leveraging the MERN stack (MongoDB, Express, React, Node.js), we can create a lightweight, "Mobile-First" digital library that puts a wealth of knowledge directly into the pockets of farmers.

The shift from traditional extension services (which are often understaffed) to a Digital Knowledge Library. This project aims to empower farmers to transition from subsistence farming to more productive, data-driven agriculture, thereby improving food security and local economies.

## PROJECT OBJECTIVES

### MAIN OBJECTIVE

To development of a Digital agricultural knowledge library for smallholder farmers in order provide a robust, scalable, and user-friendly platform for agricultural knowledge management

### SPECIFIC OBJECTIVE

1. To create a centralized repository or database, to build a searchable database using MongoDB that stores crop guides and store user detail

2. To design frontend for data access, to develop a responsive frontend using React.js that ensures the library is easily navigable on low-end smartphone screens.

3. To develop system backend, to implement a backend system using Node.js and Express that allows authorized users to Create, Read, Update, and Delete knowledge entries to keep the library current

4. To implement an authentication mechanism, Secure Information Sharing by using JWT that protects the integrity of the data, ensuring only verified experts can modify the information provided to farmers.

## SYSTEM REQUIREMENTS

### FUNCTION REQUIREMENTS

These requirements define the specific behaviors and CRUD operations your application must perform to solve the information gap for farmers

- User Authentication & Authorization:
    - The system shall allow users to register and log in securely.
    - The system must distinguish between 'Admin' (staff, trainer etc..) who can modify data and 'Viewers' (Farmers/ normal user).
- Knowledge Management (CRUD):

    - Create: Authorized experts must be able to upload new agricultural guides, including text and image references, via a Node.js API.
    - Read: Farmers must be able to browse, search, and view farming content through the React.js frontend.
    - Update: Admins must be able to edit existing entries to reflect updated agricultural practices or climate data.
    - Delete: The system must allow for the removal of outdated or irrelevant agricultural information.
- Search and Filter: Users must be able to search for specific crops or pests and filter information by category (e.g., "Cereals," "Pest Control," "Irrigation").
- Responsive Display: The interface must dynamically adjust content layout for various mobile screen sizes using React. And finally make the web in responsive web app RWA

**NON-FUNCTIONAL REQUIREMENTS**

These represent the quality attributes and constraints of your system, specifically focusing on Mobile-Driven Design (MDD).

- Mobile-First Design (Usability)
- Performance and secure
- Reliability and error handling
- Maintainability

**SYSTEM ARCHITECTURE**

The system is designed as a web-based mobile application. It utilizes React.js for a dynamic, single-page frontend experience and Node.js/Express for a robust RESTful API. MongoDB acts as the non-relational database, storing agricultural data in flexible JSON-like documents

- **Client-Server Architecture**

**Webpage** The user interacts with the mobile-first interface. When a farmer searches for a "document," React sends an HTTP GET request to the backend. And **The Server** (Node.js/Express) The server receives the request, identifies the route, and queries the database after The **Database** (MongoDB) The database retrieves the specific document and sends it back to the server and final The **Response** The server sends the data back to the client in JSON format, which React then renders onto the farmer's screen

- Database Schema (Entity Relationship Diagram)
    - User schema
        - Username
        - Password
        - Role (user, admin, staff, trainer)
    - Knowledge entity (resources)
        - Title
        - Categories
        - type
        - Description
        - Imageurl

**AUTHENTICATION & SECURITY**

- Authentication Mechanism a using JWT (JSON Web Tokens). When an Admin logs in, the server generates a token. The React frontend stores this token in local Storage and includes it in the header of every request to "Create" or "Delete" guides.
    - **Login Flow**: When a user (Farmer, Trainer, or Admin) submits their credentials, the Node.js backend validates the email and password

- **Token Generation**: Upon successful validation, the server generates a JWT signed with a secret key stored in the environment variables or use default token for demonstration
- **Client-Side Storage**: The React frontend receives this token and stores it in localStorage. This token is then sent in the Authorization header for all subsequent API requests.
- Token & Session Handling

  Because the application is built as a **Progressive Web App (PWA)**, session handling must be robust enough for mobile use, tokens are set with an expiration time (e.g., 3 days) to minimize the risk of a stolen token being used indefinitely. The server does not need to store session data in memory and ince PWAs require **HTTPS**, the token is encrypted during transmission, preventing "Man-in-the-Middle" attacks.

- Security Measures
  - Bcrypt.js: Used to hash passwords before they are saved to MongoDB, ensuring that even if the database is accessed, plain-text passwords are not exposed.
  - Middleware: Custom Express middleware is used to verify tokens before allowing access to sensitive CRUD operations.

**IMPLEMENTATION DETAILS**

The system is built using a modern full-stack approach to ensure scalability and a "Mobile-First" experience and PWA

- Frontend using a react.js for building a dynamic, component-based user interface and Tailwind CSS for responsive mobile design.
- Backend a using Node.js and Express.js to handle the RESTful API logic and server-side operations. And professional backend separates concern into different folders. This makes the code organized and readable
  - Controllers
  - Middleware, Handles security and error processing

    - Models, Defines the data structure (Schemas).
    - Routes, Handles URL paths and HTTP methods.
    - Server file, the entry point that initializes the app and connects to MongoDB
- MongoDB (NoSQL) for database to storing flexible agricultural data and user profiles.
- JSON Web Tokens (JWT) for secure session management and Bcrypt.js for data encryption.

Designed with Mobile Driven Design (MDD) principles, State Management in react hooks (useState and useEffect) in Routing use a react router and responsiveness use a grid-based layout

**SECURITY CONSIDERATIONS**

Authentication Mechanism, the system utilizes JSON Web Tokens (JWT) for secure, stateless authentication. The Process, when a user logs in, the Node.js server verifies their credentials and generates a unique, digitally signed token. Token Storage, this token is sent to the React frontend and stored in localstorage

Password Storage and Encryption, storing passwords in plain text is a critical security risk. Our system ensures that even if the MongoDB database is compromised, user passwords remain unreadable. **Bcrypt.js Hashing**, use the bcryptjs library to hash passwords before they are saved to the database

**ACCESS CONTROL IMPLEMENTATION**

The library implements Role-Based Access Control (RBAC) to distinguish between different user types like a Farmers, to granted "Read-Only" access. Admins Granted full CRUD permissions and Middleware in Express.js. Before a "Create," "Update," or "Delete" route is executed

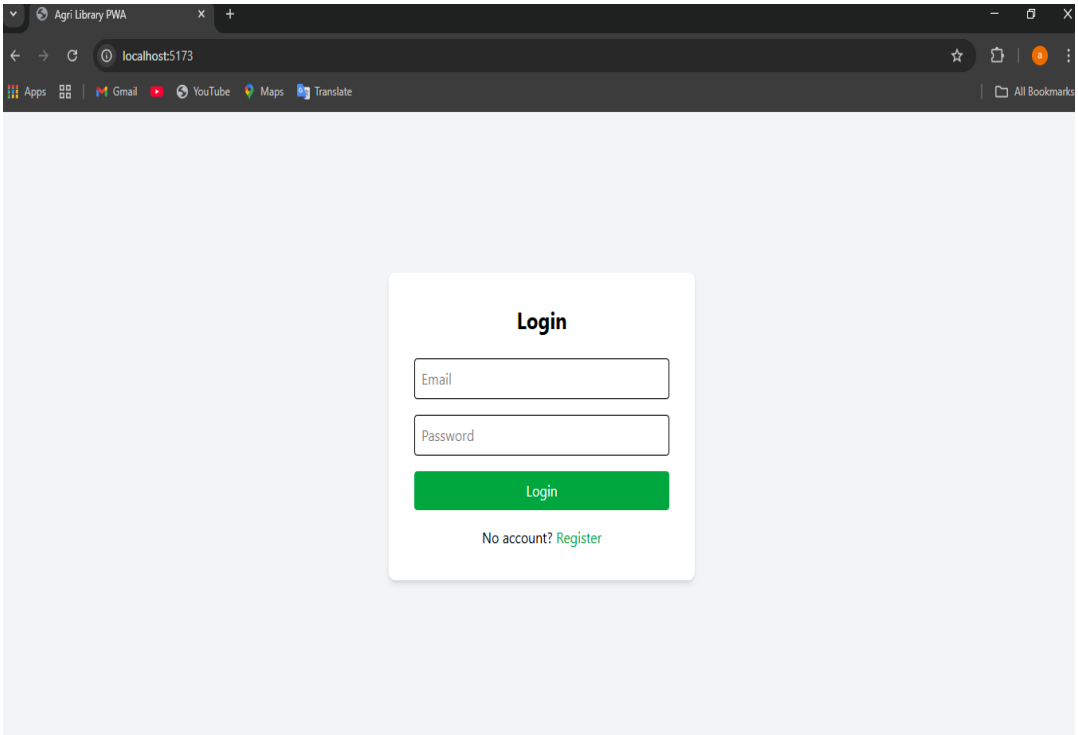
**DATA PROTECTION & INPUT VALIDATION**

To protect the integrity of the Agricultural Library, we implement several data-level security measures. In **Server-Side Validation**: Using libraries like express-validation make the porpoise of behaviour and Sensitive information, such as the MongoDB connection string and the JWT secret key,

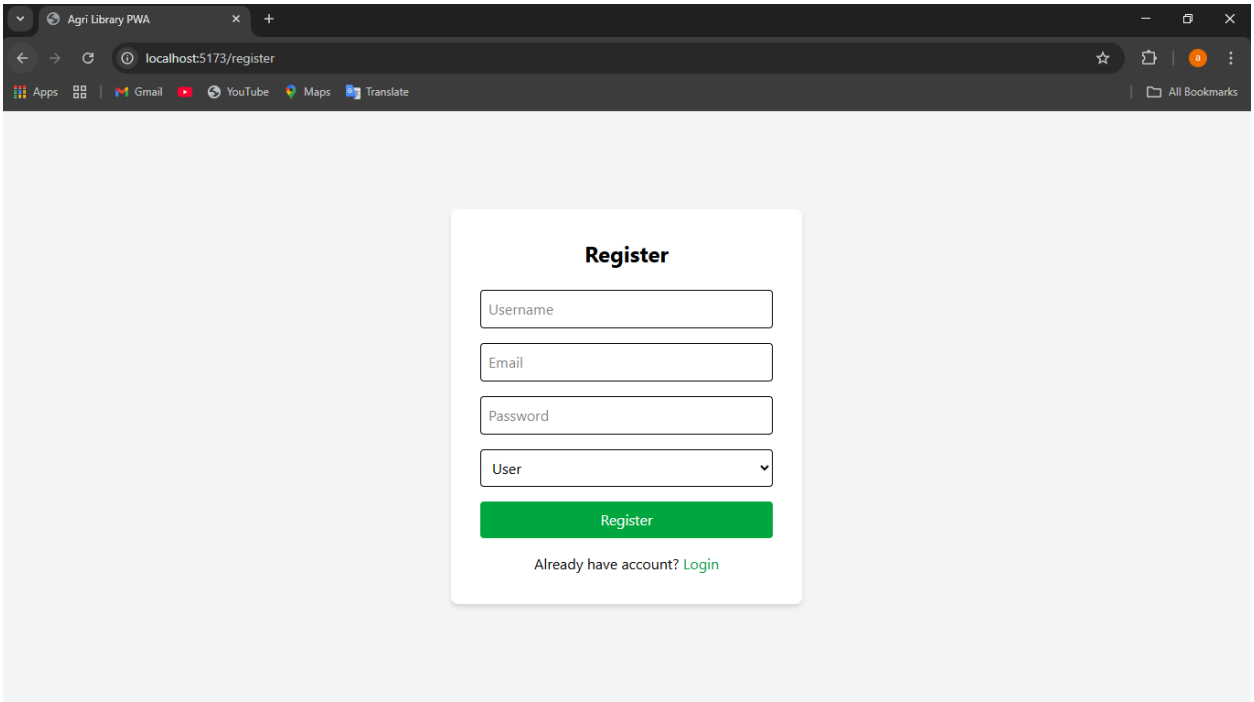
**STESTING & RESULTS**

SCREENSHOTS OF THE APPLICATION

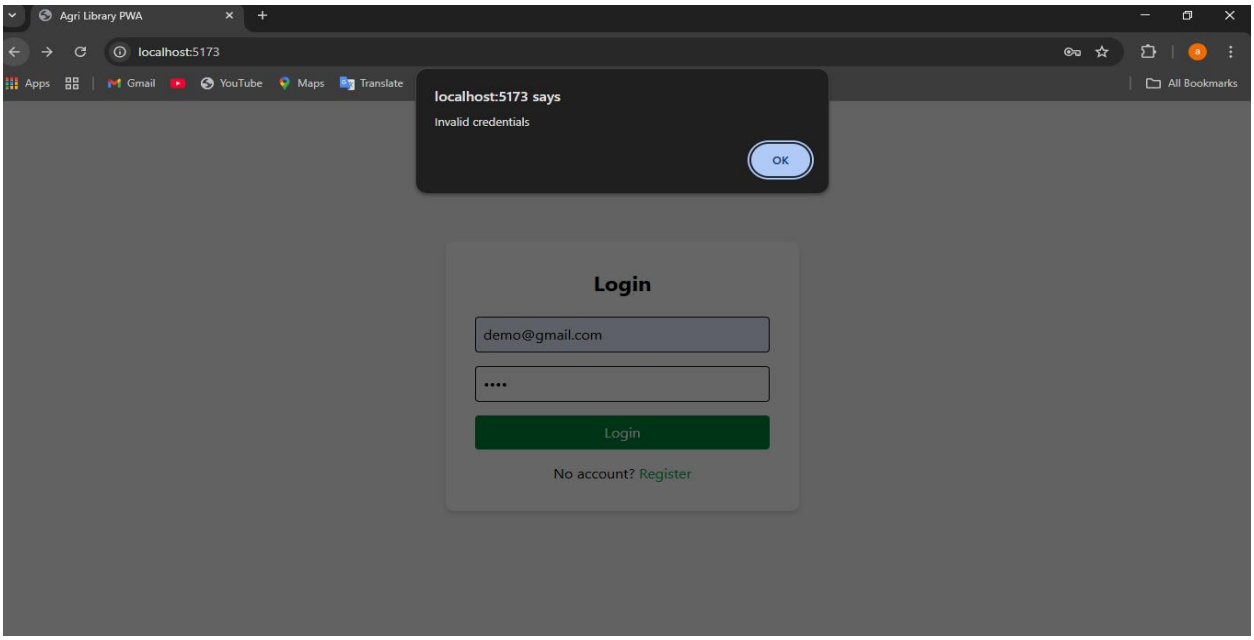
- Authentication Screens
  - Login page



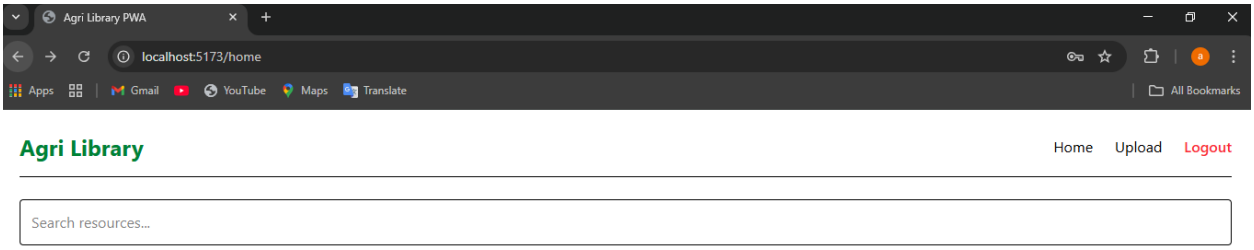

  - Register page



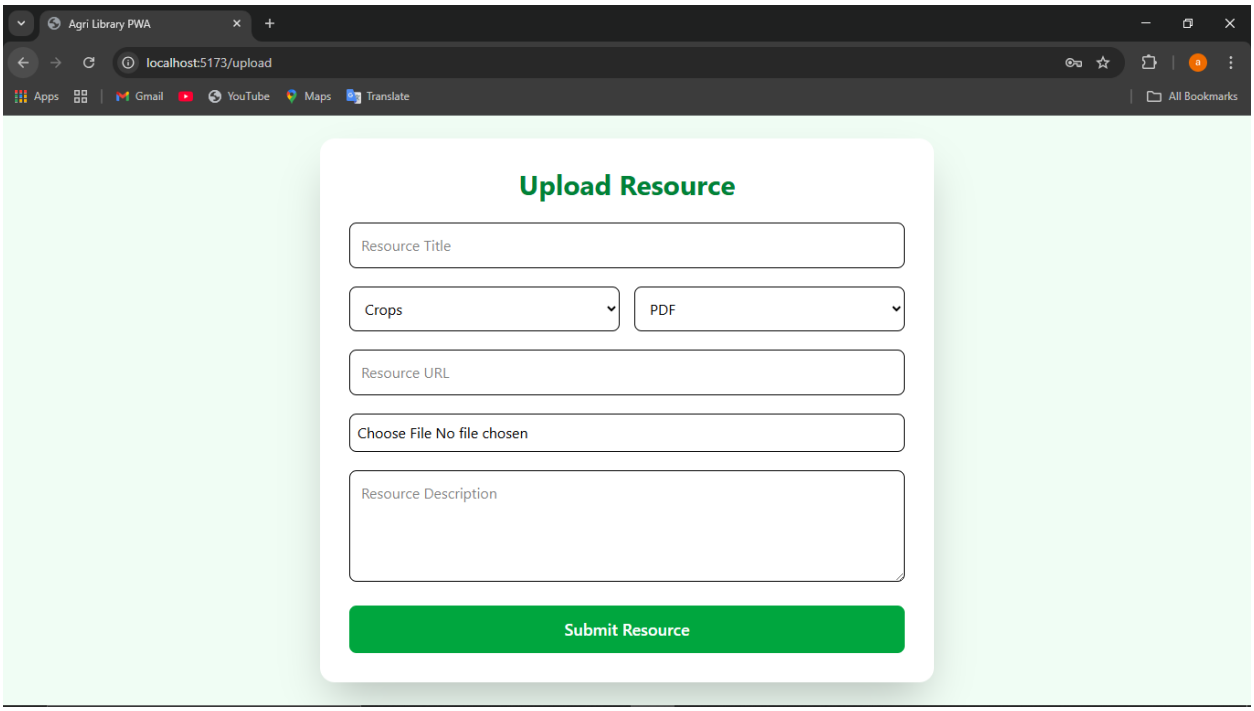

- Validation page




- Home page




- Upload page




DESCRIPTION OF INCOMPLETE FEATURES

- External API integration
- Advance image process

**FUTURE WORK**

POSSIBLE IMPROVEMENTS

Based on common challenges and emerging trends in digital agriculture, the following features could be added to enhance the library's utility

- AI-Driven Diagnostics: Integrating Artificial Intelligence (AI) and Machine Learning (ML) could allow farmers to upload photos of diseased crops
- Real-Time Data Integration: The library could be expanded from a static repository to a dynamic advisory system by integrating external APIs

SCALABILITY CONSIDERATIONS

To transition from a pilot project to a large-scale system, the following architectural improvements would be necessary

- Database Optimization
- Content Moderation Workflows

**RESPONSIVENESS (MOBILE-FIRST DESIGN)**

Used Tailwind CSS for utility-first styling to ensure a consistent, mobile-first UI. Implemented a responsive grid system (grid-cols-1) to optimize the layout for small screens, as required by MDD principles and Security Used Conditional Tailwind classes to display administrative 'Upload' buttons only to users with 'Admin' tokens.

**DEPLOYMENT / WEB APP/ PWA SEPTS**

A functional PWA setup requires a manifest.json and a Service Worker to handle offline capabilities.

- Manifest Configuration, this file allows the library to be "installed" on a farmer's phone, appearing as a native app.

    - Tailwind Integration: Use your brand colors (e.g., theme_color: "#15803d") in the manifest to match your Tailwind green-700 theme.
- Service Worker, this script caches your Tailwind-styled frontend and your agricultural guides.

To meet the functional PWA setup criteria, ensure your project includes:

- Install ability, the browser should prompt the user to "Add to App.jsx"
- Offline Support, the core UI (styled with Tailwind) should load without an internet connection.
- HTTPS: The app must be served over a secure connection (automatically provided by Render or Vercel).

**Video link**

**https://drive.google.com/file/d/1OVa58_nrnHp2XHgiOnuBaoDNbau4G-YT/view?usp=sharing**